Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Frontier and Localhost: A Survey of How Production AI Learns Outside the Weights

MIKHAIL L. ARBUZOV[1], LEE MOSBACKER[2], SISONG BEI[3], ZIWEI DONG[4], DMITRI KALAEV[5], AND ALEXEY SHVETS[6]

[1]Independent Researcher, CITY, COUNTRY (e-mail: mike.arbuzov54@gmail.com)
[2]Independent Researcher, CITY, COUNTRY (e-mail: lee.mosbacker@gmail.com)
[3]Independent Researcher, CITY, COUNTRY (e-mail: qurining@gmail.com)
[4]Independent Researcher, CITY, COUNTRY (e-mail: ziwei.dong@alumni.emory.edu)
[5]Independent Researcher, CITY, COUNTRY (e-mail: kalaevdr@gmail.com)
[6]Palo Alto Networks, CITY, COUNTRY (e-mail: ashvets@paloaltonetworks.com)

Corresponding author: Mikhail L. Arbuzov (e-mail: mike.arbuzov54@gmail.com).

TODO(authors): funding statement, e.g., "This work received no external funding." A. Shvets is employed by Palo Alto Networks; the views expressed are the authors' own.

ABSTRACT Deployed AI systems change through instructions, skills, memories, tools, and orchestration long before their model weights are updated. These changes accumulate experience, but their persistence alone does not establish that the system is improving. This survey examines how 256 production and research systems turn experience into lasting changes outside the weights. We distinguish two decisions: whether an edit improves the deployment that produced it, and whether there is enough evidence to share it more widely. Together, these decisions define a local improvement loop and a promotion loop. Automatic acceptance tests are present in nearly every system with failure-driven local adaptation, while failure capture and attribution remain less developed. Promotion is more fragmented: the surveyed systems provide evaluation, shared artifact pools, lineage, access control, and rollback in different combinations, but none documents their integration into governed promotion across organizations. The feedback available to a deployment helps explain this pattern. Exact checks, observed outcomes, judgments, and delayed rewards support different acceptance rules and expose different limitations. In every case, an accepted edit is supported only for the measure and task distribution on which it was evaluated. We use this observation to develop five conditions for artifact-layer optimization, a classification of feedback and gate types, and four properties for auditing cross-organization promotion. The resulting framework identifies what existing systems can already automate and where new evidence or human judgment is still required.

INDEX TERMS AI governance, agent memory, agent skills, evaluation, large language models, LLM agents, prompt optimization, self-improving systems, survey

I. INTRODUCTION

A coding agent repeatedly uses the wrong naming convention in a repository. An engineer adds a line to its instructions, and the next session follows the convention. The model has not changed, but the deployed system has. Similar changes happen when an assistant corrects a stored preference, acquires a reusable skill, or learns which tool to call. Much of the experience that makes an AI system useful in one setting now accumulates in these persistent artifacts outside the model's weights.

We call this layer the **scaffold**. It includes instructions, skills, memory, tools, orchestration, and the evaluation and governance rules that control their use. In the language of compound AI systems [1], it is the part surrounding the model that a deployment can inspect and revise without retraining it. The distinction matters because the two parts change on different schedules. A team can revise an instruction or tool binding as soon as a problem is understood, while a new model requires a separate training, evaluation, and release process.

The ease of making a change also makes it easy to accumulate changes whose value is unknown. A rule added after one incident may prevent a recurring error, conflict with another rule, or merely make the prompt longer. A corrected memory may remain in the history while the system continues to retrieve the outdated entry. Even a demonstrably useful change can become harmful when copied into a deployment with different tasks or conventions.

These situations raise two questions. What evidence justifies keeping a local change? What additional evidence justifies sharing it? This survey uses those questions to examine 256 public production and research systems. The first defines a **local loop**: identify a recurring failure, propose a change to the responsible artifact, and test whether the change helps. The second defines a **promotion loop**: decide whether the change should be adopted by another project, team, or organization, and preserve enough history to reverse that decision.

The systems surveyed have developed these loops unevenly. Automatic tests for accepting a local edit are present in nearly every system that adapts from failures. The steps that should supply those tests with well-founded candidates are weaker: failures are often not retained in a usable form, and a failed output rarely identifies which instruction, memory, skill, or tool binding caused it. Meanwhile, production platforms have built substantial versioning and release machinery without any process that proposes changes from deployment experience.

Sharing adds a further limitation. A change that improves one deployment has been evaluated against that deployment's tasks and measure of success. Another deployment may need different behavior. Shared formats and registries make artifacts portable, but they do not establish that an artifact will help its recipient. Some surveyed systems check transferred changes within one operator. None documents an automatic local improvement loop connected to promotion across organizations with versioned lineage, access control, and rollback.

The feedback available to each system helps explain these differences. A compiler or test suite can check a candidate immediately. A booked appointment provides a later, noisier outcome. A writing task may depend on a judge, whose preferences can drift. A research decision may take months to evaluate. These signals support different kinds of automation, and none establishes an improvement beyond what it actually measures.

The survey develops a framework for reading this evidence. We first define the two loops and the conditions under which artifact maintenance constitutes optimization. After describing the corpus and scoring method, we examine local adaptation, the feedback that constrains it, and the evidence needed for promotion. The later sections consider failures that arise when these checks are missing, specify the unassembled architecture for governed sharing, and connect the findings to the research lines that produced its components. The evidence consists of published methods and public documentation; the survey introduces no new experiments. Its scores and classifications are released for inspection.

II. FROM A LOCAL FIX TO A MEASURED IMPROVEMENT

Adding a naming rule after an error is a reasonable response. To learn whether it works, however, the team must run the agent again on relevant tasks and compare the results. If the change helps, it should persist. If it causes regressions, the team needs a way to reject or reverse it. Repeating this process turns a growing collection of fixes into a procedure for improving the deployment.

A. THE LOCAL LOOP

The procedure starts with a record of failures: rejected answers, user corrections, failed tool calls, evaluation regressions, or unsuccessful tasks. Repeated failures are grouped into modes and prioritized by frequency and severity. For each selected mode, the system tries to identify the artifact responsible. A stale memory entry calls for a different intervention from a missing tool argument or an ambiguous instruction.

We call this step **credit assignment**. It links an observed failure to an editable part of the scaffold, or **coordinate**. The system then proposes a candidate edit and evaluates it against the current scaffold. The acceptance test is the **gate**. An accepted edit is retained as a versioned artifact; subsequent evidence of regression can trigger rollback. Fig. 1 shows the complete local loop.

The evaluation must be tied to the deployment's actual workload. A deployment serves a particular distribution of users, tasks, and conventions, which we call its **patch**. Its **patch loss** measures failure on that distribution. Depending on the application, this may be a task-failure rate, correction rate, retrieval error, or another operational measure. A gate can establish an improvement only to the extent that its evaluation estimates this loss.

B. THE SECOND DECISION: WHETHER TO SHARE THE FIX

Suppose the naming rule improves the original repository. The team may now consider adding it to a shared template. That decision changes the population on which the rule will act. Other repositories may follow a different convention, and the local evaluation says little about them.

The promotion loop therefore asks for evidence at the receiving scope. We call this **receiving-side evaluation**: a check conducted on the tasks of the deployment that would adopt the edit. The **evidence radius** is the scope for which the available evaluation supports the change. A local success does not automatically enlarge that radius. Section VI develops the scopes and examines systems that already transfer artifacts between them.

C. FIVE CONDITIONS FOR ARTIFACT-LAYER OPTIMIZATION

We use **artifact-layer descent** for the form of maintenance that connects recurring failures to measured improvements through five conditions:

1. **A measurable patch loss:** the deployment has a defined measure of success or failure.
2. **A failure log:** relevant failures are retained as evidence for later changes.
3. **Credit assignment:** failures can, at least sometimes, be traced to the scaffold coordinates responsible.
4. **A candidate edit:** the process proposes a change to those coordinates.
5. **An acceptance gate:** an edit is retained or promoted only after a check of whether it lowers the relevant loss.

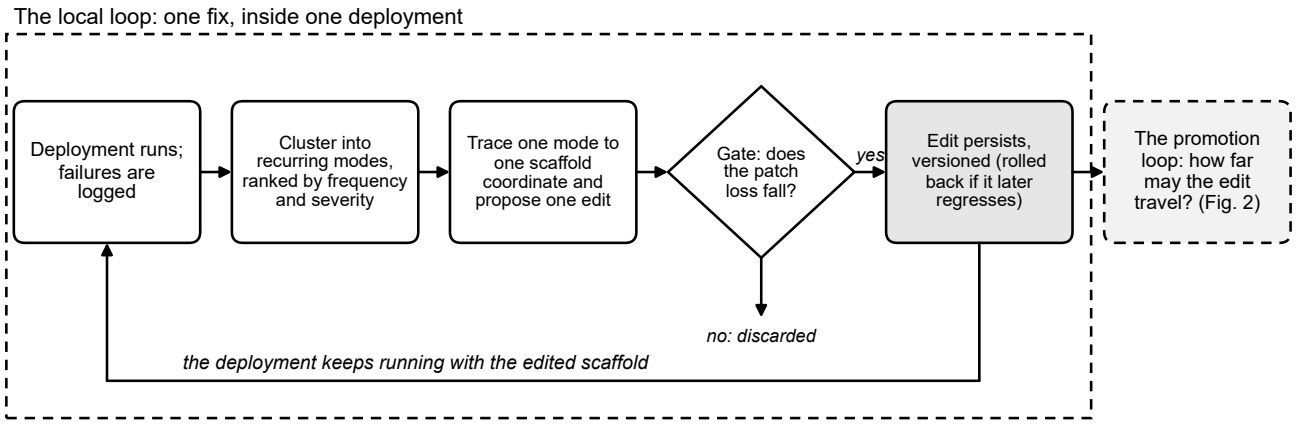


FIGURE 1. The local improvement loop. Recurring failures guide a proposed scaffold edit. An acceptance gate tests the change before it persists, and later regressions can trigger rollback. Sharing the accepted edit requires a separate promotion decision (Fig. 3).

The model weights remain fixed during this process, and the edits persist beyond the current interaction. These conditions distinguish three recurring forms of maintenance. In *patchwork*, an engineer adds a rule after an incident without measuring the result. In *random artifact search*, a system tests many variants and keeps a winner without tracing failures to their causes. Artifact-layer descent uses that trace to direct the proposed change and tests it before acceptance. Where the conditions are only partly met, the optimization interpretation is correspondingly limited.

For deployment D, write the patch loss as

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,,$$

where θ_M denotes the fixed model weights, θ_s the editable scaffold, and ℓ the operational loss. A proposal process uses the failure history to produce an edit z_t. There is no analytic gradient through an instruction file or memory entry. Instead, the gate estimates the change in loss and accepts when that estimate clears a required margin:

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda\Omega(z_t) \le -\tau_t.$$

Here $\Omega \ge 0$ penalizes edit complexity and promotion breadth, and $\tau_t \ge 0$ is the acceptance margin. An evaluation that blocks a regressing merge is a practical instance of this rule. A reviewer or governance process can also instantiate the gate by requiring stronger evidence for a broader or more complex change.

Appendix XI gives the update rule and a one-step descent inequality under assumptions on the gate's estimation bias. With additional coverage, compactness, and summable-bias assumptions, improving edits eventually stop clearing the gate. This is **gate stability**, not a claim of global optimality. A limited proposal process or biased evaluator can leave useful changes undiscovered or rejected.

D. WHO PROPOSES AND WHO DECIDES

The author of an edit and the authority that accepts it are separate. An agent may draft a revision that a person approves; an engineer may write a revision that an automatic evaluation rejects. We classify the gate by who makes the acceptance decision:

- **Auto:** a program applies a check, including a verifier, evaluation threshold, or language-model judge [2]. Whether that judge has been calibrated against people is a separate question, which Section V takes up.
- **Human:** a person decides whether to accept the change.
- **None:** no acceptance decision is made, either because the loop is absent or because every proposed edit is retained.

A pair such as [Auto × Human] denotes an automatic local gate followed by a human promotion gate. These labels describe the decision mechanism. They do not, by themselves, establish that the decision is well supported.

The loss definition imposes two limits on every such loop. First, an accepted edit is evidence of improvement only on the measure and tasks evaluated. Promotion must preserve that meaning or establish the improvement again. Second, the loop optimizes a defined objective. It does not discover a better objective simply by accumulating edits. Revising the evaluator changes the objective itself, a complication we return to in Section VII-B.

III. CORPUS AND METHOD

We read each system's public method or documentation for four things: the persistent artifact it changes, the feedback that prompts the change, the decision that accepts it, and the scope in which the result is used. This allows systems with different names and application domains to be compared by their behavior.

A. SCOPE AND SELECTION

The corpus contains 256 unique systems from January 2023 through the October 2026 cutoff. The first search pass iden-

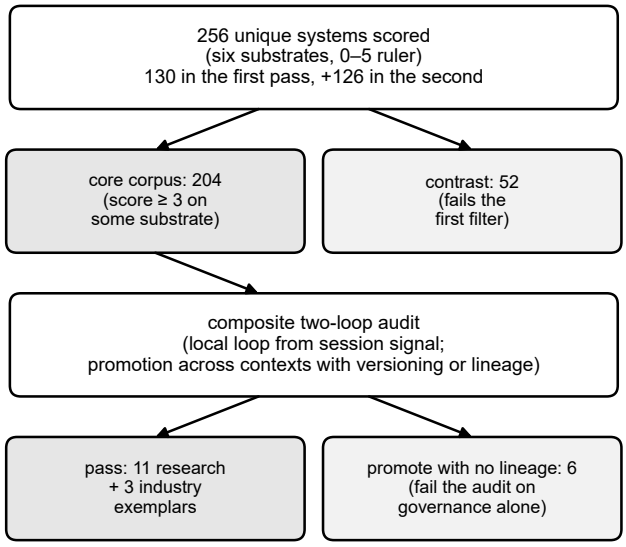


FIGURE 2. Corpus selection. The 256 systems occupy 269 scored rows. The first filter separates core from contrast systems; the two-loop audit then checks persistent local adaptation and promotion with lineage. The audit includes the additional paper-based classifications described in the text.

tified 130 systems by May 2026; the second added 126. Sources include research papers, public repositories, product documentation, and supported production reports; the search families, evidence tiers, and inclusion rules are recorded with the released data (Supplementary Material S2).

A system enters the survey if it adapts a foundation model through a persistent artifact updated from deployment signal, or if it is publicly described as self-improving, scaffolded, or agentic. Including the latter group provides a contrast set: the survey can test whether those descriptions correspond to persistent adaptation. Systems whose only update mechanism is weight-level training are excluded, as are systems without public evidence of release or deployment.

We apply two filters. The first admits a system to the **core corpus** if it scores at least 3 on at least one scaffold substrate under the rubric below. Of the 256 systems, 204 form the core corpus and 52 the contrast corpus. A system may have one integrated row or several substrate-specific rows, so the 256 systems occupy 269 scored rows.

The second filter is a **two-loop audit**. It asks whether session feedback changes a persistent artifact and whether that artifact is subsequently promoted beyond the deployment that produced it, with versioning or lineage. Applied to the core systems and the additional paper-based classifications (Supplementary Material S2), it admits 11 research systems and 3 industry exemplars. Fig. 2 summarizes the selection.

The research systems passing the audit are NanoResearch, SkillOpt, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoManual, and CASCADE. The industry exemplars are Sierra Agent OS 2.0, Cognition Devin, and Letta's Context Repositories. SkillOpt is classified from its paper and is not a row in the 256-system score table. The remaining paper-based systems are identified in the data notes (Supplementary Material S2) and the audit's boundary cases in the contrast set (Supplementary Material S4).

Passing this audit does not establish governed sharing across organizations. All passing systems promote within one operator, into a shared bank or archive that later runs draw from, or across the agents and sessions of one deployment. Letta passes on the lineage of its shared-memory updates even though those updates have no acceptance gate. Six further systems promote across contexts without lineage on the promoted artifact; their treatment under relaxed audit rules is reported separately.

B. SUBSTRATES AND SCORING

The scaffold has six substrates: instructions, skills, memory, tools, orchestration, and governance. These are the possible targets of a proposed edit. Table 1 describes them; the episodic, semantic, and procedural memory categories follow Sumers et al. [3].

The common scoring scale runs from 0 to 5. At 0 an artifact is ephemeral. At 1 it persists locally; at 2 it persists with tool attachment but without feedback. At 3 it either spans scopes without a gate or is updated from feedback by a person. At 4 a program updates it from feedback. The score-5 requirement depends on the substrate, as specified in the table.

This last distinction is important when interpreting the scores. Instructions and skills require both loops to reach 5. Orchestration requires a topology updated from deployment feedback by a loop outside the agent's own execution. Memory, tools, and governance are scored for their corresponding governance capabilities, so a registry can score 5 without generating local edits. Integrated rows measure breadth and depth of adaptation, not closure of either loop. The scale therefore supports substrate-specific comparisons; a top score is not a universal certificate of complete optimization.

Table 2 gives the corpus distribution. Research systems hold the top marks on skills, memory, and the integrated row. Production platforms and standards hold them on tools and governance. No system reaches the top score across all six substrates, and none reaches it on instructions or orchestration.

For the 126 most recently added systems, gate types, feedback classes, and governance properties are recorded as explicit columns of the released table, so counts based on those columns apply to those systems. For example, 89 of their 105 core systems close the local loop under a program. This is not a gate count over all 256 systems.

C. EVIDENCE AND LIMITATIONS

The evidence ranges from peer-reviewed studies and open-source artifacts to vendor claims. Every row of the released table carries its source and a one-line justification, and the rows for the 126 most recently added systems also carry an evidence tier. Peer-reviewed papers and open-source artifacts receive the greatest weight. A production claim must name a benchmark or integration target, and a blog or interview

TABLE 1. The six scaffold substrates, what each holds, and what a system must do on it to score 5.

Substrate	What persists	A score of 5 requires
S1 Instructions	Project, team, or organization rules attached to a workspace	Versioned rules promoted under an automatic gate
S2 Skills	Procedures, as code or text, loaded on demand	A skill bank that edits itself on failure, test-gated, versioned, and promoted
S3 Memory	Experience kept across sessions: episodic, semantic, procedural	Provenance, versions, and a correction path for every entry
S4 Tools	The vetted action surface and its schemas	Evaluation-gated tool versions; a curated bundle as the product
S5 Orchestration	The multi-role graph and its routing	Topology updated from deployment feedback by a loop outside the agent's own run
S6 Governance	Versioning, promotion, audit, rollback	Staged promotion with evaluation thresholds, an audit log, and rollback

TABLE 2. Substrate maturity across the 256-system corpus. Column n counts scored rows on each substrate, Mean is the average score, and Score-5 systems reach the substrate's top mark; the integrated row counts systems scored across several substrates as one row. Of the 126 most recently added systems, 105 are core and 32 are skill systems; 89 of those core systems close the local loop under a program, and none of the 126 scores 5 on any substrate.

Substrate	*n*	Mean	Score-5 systems	What the corpus holds on it
S1 Instructions	15	3.07	(none)	Rules persist across IDEs and agents and nothing measures them; no system closes both loops; ceiling 4
S2 Skills	50	3.34	SAGE, COSPLAY	The local loop closes under a program in the research systems; the production repositories merge by hand
S3 Memory	50	3.06	Cuadros et al.	The ledger ships in products; the correction a gate would enforce is absent
S4 Tools	20	3.65	MCP, Harvey, Hippocratic AI	The most mature production substrate; registries govern admission, not evaluation
S5 Orchestration	37	3.08	(none)	Topologies change under an evaluation gate; one structurally gated instance; no two-loop system with lineage
S6 Governance	45	3.02	Braintrust, Vellum, LangSmith Hub, AGENTS.md under the Agentic AI Foundation (AAIF)	Prompts-as-code mature; registries add lineage and rollback; no local loop generates a candidate
Integrated	52	3.81	NanoResearch, AutoAgent, SkillRL	Breadth and depth of adaptation, not the loop; reached only by research systems without enterprise governance

is counted only when stronger evidence corroborates it. Production claims are not pooled with peer-reviewed evidence in corpus-level statistics; vendor-based exemplars are identified as such.

One author scored all systems. Ambiguous cases were rounded down, and half-scores were used only when one criterion was clearly met and another partly met. Independent re-rating remains future work. The classifications should be read as an auditable interpretation of public evidence, not a complete inventory of private production behavior.

We also test the audit under alternative boundaries: counting rollback as sufficient promotion governance, allowing promotion without governance, and admitting loops that persist only optimizer state. Membership changes under these relaxations, but the principal structural findings remain: [Auto $\times$ Auto] stays well populated, the human-local cells hold only curation and registries, Letta alone passes with an ungated promotion, and no relaxation adds governed promotion across organizations (Section VIII). The sensitivity analysis is released with the data notes (Supplementary Material S2) and the full score table with the released table (Supplementary Material S1).

IV. WHAT LOCAL ADAPTATION ACTUALLY CLOSES

A system can detect a failure, explain it, and write a plausible correction without ever establishing that the correction helps. SkillForge illustrates this distinction. Its judge agrees with people more than nine times in ten and identifies replies that should be rewritten. Those replies drive updates to the skill, but the updated skill is committed without an acceptance test [4]. The calibration concerns which replies are inadequate. It does not measure the effect of the resulting skill changes.

SkillRL exposes the same separation through a different mechanism. A success-rate threshold triggers proposals for new skills, and the skill bank accepts each proposal uncondi-

tionally [5]. Both systems respond to experience, yet neither has the local acceptance gate defined in Section II. This is why the survey records who accepts an edit separately from who generates it or which signal triggers it.

Among the 105 core systems most recently added to the corpus, automatic local acceptance is nevertheless the dominant pattern: 89 use a program to retain or reject changes. Looking inside those loops reveals a less even picture. Evaluation is comparatively mature, while the failure record and the link from a failure to the responsible artifact are often incomplete.

A. FINDING THE RIGHT THING TO CHANGE

A useful failure history must retain what went wrong in a form that supports later comparison. Production memory more often records preferences, successes, or conversation history. Only one field report in the corpus describes a store of dead ends intended to prevent their recurrence, and that evidence comes from a single self-reported deployment [6]. The persistent store may therefore be extensive while the particular record needed for failure-driven adaptation is missing.

Even when failures are retained, nearly every system uses them in arrival order. Few explicitly choose which recurring modes deserve attention. Task-CoEvolve's variance-weighted task sampling and AutoSaddler's mini-batch failure signals are exceptions [7], [8]. They weight the experience used for the next update rather than letting the most recent session determine it. The general loop described earlier would prioritize both recurrence and severity; this remains uncommon in the surveyed implementations.

The next step is harder. A failed output does not reveal whether its cause was an instruction, a retrieved memory, a skill, or the route through a group of agents. Harness-optimization papers repeatedly identify this attribution problem as the bottleneck [9]–[13]. An evaluation of harness optimizers reaches the same conclusion: by downstream score alone, a harness optimizer cannot be told from trial and error [14].

Existing solutions usually operate within one substrate. Meta-Harness preserves candidate harnesses and the record of which edits helped [11]. SkillsVote attributes outcomes to the skill, exploration, or environment before admitting an update [13]. SkillOpt-Lite keeps only the attributes that recur across rollouts, and its ablation finds this consensus mechanism load-bearing [15]. These methods give the proposal process more useful evidence. None solves the general attribution problem across all six scaffold substrates.

B. TESTING THE CANDIDATE

Research systems commonly generate candidates by reflecting on trajectories. In production prompt platforms, an engineer writes the candidate and session feedback never triggers one, while public skill repositories depend on contributors. A field study of five such repositories finds that every substantive edit was committed under a named human account and merged by a person, most carrying an AI co-author trailer [16]. Where proposal generation depends on a person noticing a problem, the evaluation machinery can run automatically while the improvement process itself remains intermittent.

On skills, SAGE and COSPLAY close both local and promotion loops inside a research environment [17], [18]. SkillChain adds a human reflection decision after the judge, on which refined skills are deployed [19]. SkillRL and SkillForge, described above, leave the proposed changes unchecked. These differences would be obscured by classifying all three simply as systems that learn from feedback.

Orchestration has followed a similar progression. Early systems searched for agent topologies under an evaluation score [20], [21]. More recent systems evaluate proposed topology changes before keeping them; TOCOMAS also requires a structural invariant [22]–[24]. OptiMAS is an exception: its optimizer reads trajectories and validation scores to write the next configuration, but the score informs the edit rather than supplying a keep-if-better decision [25]. No orchestration system closes both loops with lineage on the promoted artifact.

The most widely deployed artifact remains a useful counterpoint. Instruction files are present across Claude Code, Cursor, AGENTS.md [26], Windsurf, and Continue.dev, and adoption studies find them in roughly a fifth to a quarter of coding-agent projects [27], [28]. Their prevalence does not settle their effectiveness. One controlled study reports higher inference cost without improved task success [29]. Another toggles the root AGENTS.md in paired runs on 124 pull requests and reports about 29% lower median wall-clock time and 17% fewer median output tokens, with total tokens approximately unchanged; it does not measure correctness [30]. The studies measure different outcomes and disagree on cost. Neither establishes a general task-success benefit from accumulating instruction files.

Human local gates appear only in curation. VerificAgent uses fact-checked, frozen memories [31], and in SkillChain a person decides, at a reflection gate, which refined skills are deployed. Prompt platforms and registries without a failure-driven proposal process have no local gate in this classification. Their checks act on release or admission to a shared pool, which belongs to promotion.

C. GATHERING ENOUGH EVIDENCE TO ACCEPT

An acceptance test can itself be too noisy to support a decision from one run. Several newer systems address this directly. SAGE-gate, a statistical acceptance method unrelated to SAGE, uses paired per-item comparisons before committing an edit [32]. VaG requires three heterogeneous critics to pass [33]. AutoSaddler uses mini-batch failure signals in validation-based selection [8].

These mechanisms should not be conflated. Averaging k independent noisy scores reduces the variance of their mean by a factor of $1/k$. Requiring agreement among k independent checks, each with false-acceptance probability p, gives the idealized probability p^k. The independence assumptions matter, and actual critics need not satisfy them. Appendix XII

distinguishes the gate families and their roles. The shared development is that acceptance increasingly depends on a body of evidence rather than one favorable observation.

D. MAINTAINING THE ARTIFACTS THAT SURVIVE

The evaluation of a new edit also depends on the artifacts already present. Stale or conflicting skills can cause failures that obscure the effect of the candidate under test. Ratchet reports no gain from model-written skills over a no-skill baseline until lifecycle hygiene is applied, while human-curated skills provide a double-digit gain [34]. Eleven surveyed skill systems now treat removal as an operation that must pass a check, just as admission does [35]–[38] (Supplementary Material S6).

Memory products have made substantial progress on a different part of maintenance: recording an entry's history. Letta's Context Repositories use git, Mem0 retains per-memory history, and Zep represents facts bitemporally [39]–[41]. These records can explain what the system remembers and how that state arose.

History does not guarantee correction. A study of five production backends finds that all five still return facts that were revoked [42]. The record contains the change, but retrieval does not reliably enforce it. Research systems likewise divide between provenance and write controls [43]–[46] and verifier-gated failure correction [47], [48]. Cuadros et al.'s governed collaborative memory combines provenance, versioning, and correction, earning the top memory score [49].

Reversal has its own limits. SkillSmith and SkillAudit support edit-level rollback [36], [50], and EvoUndo rejects edits that cannot be undone [51]. VaG finds that reversing a contaminated skill recovers little once other skills have been derived from it [33]. The consequences of an edit may therefore outlast the edit itself. Section VII returns to this problem when considering promotion and artifact descendants.

E. A COMPLETE LOCAL EXAMPLE

SkillOpt provides a concrete instance of all five conditions [52]. Its persistent state is one markdown skill document used by a frozen agent. Rollout and reflection batches supply the failure evidence. A consensus filter helps attribute recurring errors, and bounded add, delete, and replace operations propose changes. A held-out exact-match split supplies the loss, and a candidate is accepted only when its score strictly improves.

Across six benchmarks, seven target models, and three harnesses, SkillOpt is best or tied in every cell reported by its authors. Its follow-up, SkillOpt-Lite, removes a rejected-edit buffer and slow update schedule while retaining the held-out gate and consensus filter; it matches or exceeds the original [15]. This comparison supports the importance of the gate and this form of credit assignment in that setting, while identifying the removed controls as stabilizers rather than necessities.

SkillOpt also evaluates transfer across models, harnesses, and benchmarks. Those experiments measure how far the learned artifact remains useful. They are distinct from the acceptance decision that admits an edit to the shared skill bank. This distinction becomes central once the same local procedure is used in settings where correctness is harder to measure.

V. WHAT THE FEEDBACK ALLOWS A GATE TO KNOW

A coding agent changes a repository rule and receives a test result within seconds. A scheduling assistant changes its booking procedure and learns later whether an appointment was made. A marketing assistant changes its tone instructions and receives a reviewer's preference. A research assistant changes its strategy and may wait months to learn whether the direction was productive.

All four systems can propose a persistent edit. What differs is the evidence available for accepting it. The signal's timing, exactness, and dependence on judgment determine how well a gate can estimate the change in loss. We divide the surveyed systems into four feedback classes, summarized in Table 3.

The classes are assigned by the constraint that binds first. A long delay makes a signal C4 even when the eventual outcome is objective. Among timely signals, the absence of an independent ground truth makes the signal C3. Among timely objective signals, repeatable exact checks distinguish C1 from noisy observed outcomes in C2. A system may use several signals, but its operative acceptance decision must be read against the signal that supports it.

A. EXACT CHECKS: C1

A compiler, proof checker, schema validator, or exact-match answer key can decide a specified property immediately and repeatably. This makes an automatic gate straightforward to implement. AlphaEvolve scores candidates through a cascade of exact evaluators [53], and SkillOpt uses held-out exact-match evaluation [52]. DGM makes a narrower acceptance decision: it retains self-modifications that pass a sandboxed compile-and-edit functionality check on ten benchmark tasks. The broader benchmark score weights parent selection from the archive rather than determining retention [54].

Exactness applies to the check, not to the deployment as a whole. Repeating a deterministic check on the same output yields the same answer, but an agent produces stochastic outputs, and a finite suite samples only part of the task distribution. Estimating the change in patch loss still requires accounting for that variation. Passing the suite also establishes only the properties the suite tests.

Two problems remain even with a reliable checker. A failed test may not identify the instruction or skill that caused the failure, and repeated optimization against the same suite can overfit it. Cheap acceptance can also allow the artifact pool to grow without sufficient pruning. Exact feedback makes evaluation easier; it does not supply attribution or lifecycle management.

TABLE 3. The four feedback-signal classes. Reading across a row: what the gate receives, what it can therefore be, and what it cannot fix.

Class	The signal the gate receives	What the gate can be	What stays hard	Corpus examples
C1 exact	A check that returns pass or fail, or an exact score, at once and repeatably: tests, compilers, answer keys	An automatic check; one run per case, repeated only to sample the task mix	The lift is bounded and specific, an improvement in what the check measures on the cases it ran; the check names the failed output, not the coordinate that caused it; a check that is also the target can be gamed	DGM, AlphaEvolve, SkillOpt
C2 observable outcome	A real result one episode later: a booked slot, a merged pull request, a closed ticket; noisy and confounded	An automatic threshold over batched episodes; a person at promotion	The outcome says *that* the episode failed, not *why*; the channel can be poisoned	Devin, Sierra, SAGE, SkillRL, PACE, Voyager
C3 judged	A rater, a rubric, or a model judge; no ground truth	A judge calibrated against people; a person at promotion	The rubric is itself a scaffold coordinate, so the gate judges edits to itself; the proxy gets optimized past the point where it tracks quality	SkillForge, CASCADE, skills-repository review
C4 delayed	An outcome weeks or months later, once per many edits	No per-edit gate; proposals are pruned and milestones stand in	Credit across many edits; drift; the loop cannot be told from random search	NanoResearch, CORAL, the AI Scientist

B. OBSERVED OUTCOMES: C2

A booked appointment, closed ticket, merged pull request, or successful tool call provides an observable result after an episode. The result is meaningful but may be noisy or confounded. A user can accept an incorrect answer or reject a correct one, and a merged pull request reflects more than the agent's technical performance.

Devin reads merged pull requests as feedback [55]. SAGE measures task success in AppWorld [17], and PACE uses failure signals from a deployed small model [56]. SkillRL uses a success-rate threshold only to trigger proposals, without testing whether the proposed skills should remain [5]. The same kind of outcome can therefore support different operations in different systems.

Simulated outcomes, such as those in Voyager's game world or SAGE's AppWorld, remove the user confound and can be sampled repeatedly. They are treated as a distinguishable case within C2. For observed outcomes more generally, an automatic gate needs a positive margin and enough episodes or independent evidence to absorb noise. Even then, the result usually reveals that the task failed more readily than why it failed.

Promotion is harder because another deployment may differ in users, tasks, and acceptance behavior. A sender's successful outcome is not an evaluation of the recipient. This class also depends on channels that can carry poisoned memories or injected tool instructions, connecting the reliability of feedback to the integrity of its source.

C. JUDGMENTS: C3

For style, persuasion, design, or other preference-dependent tasks, a gate relies on a person, rubric, reference comparison, or model judge. There is no independent exact answer to the preference being measured. CASCADE uses a human-and-agent gate for memory consolidation [57], and a reviewer approves promotion into Anthropic's skills repository. SkillForge uses its calibrated judge earlier in the process, to identify replies that motivate a rewrite [4].

A program can make these decisions, but its score must be calibrated against the human judgment it is intended to represent before the score is treated as evidence of that preference. The gate labels deliberately do not record this distinction: an uncalibrated model judge is still an Auto gate, though the meaning of its acceptances is less established. SkillForge, SkillChain, and SkillCoach report calibration measurements [4], [19], [58]; no other surveyed judge does.

Optimization introduces another risk. A scaffold can become better at satisfying the judge while becoming worse at the underlying task. Reward-model research measures this form of proxy overoptimization [59]. Human evaluation is not fixed either: Shankar et al. find that evaluators' criteria change as they grade more outputs [60]. If the system also edits the rubric, it changes the instrument used to assess subsequent edits. This **reflexive gate** is developed in Section VII-B.

D. DELAYED AND SPARSE OUTCOMES: C4

Research programs and open-ended discovery may produce meaningful outcomes only after weeks or months and many intervening edits. The AI Scientist illustrates the broad setting by generating ideas, conducting experiments, and writing a paper [61]. The eventual outcome cannot usually be assigned to each preceding scaffold change.

Systems in this class consequently use intermediate decisions. NanoResearch merges skills by semantic compatibility [62]. CORAL promotes findings into shared memory on a heartbeat [63]. These operations can prune or consolidate

proposals, but they do not estimate each edit's contribution to the eventual loss. AlphaEvolve's archive diversity does not make it a delayed-signal example: its candidates have already been scored by exact evaluators, so archive selection is a search strategy within C1 [53].

For C4, the descent interpretation applies only to whatever proxy gate is available or to an evaluation at the longer horizon. Temporal credit assignment remains unresolved, and the environment can change before the result arrives. Outcome gains alone are also insufficient to distinguish directed improvement from a successful search, a limitation emphasized in the evaluation of harness optimizers [14].

E. CONSEQUENCES FOR THE TWO LOOPS

The classification yields three expectations that can be checked against the corpus. First, local automatic acceptance is most directly supported by exact checks and observed outcomes. Judged acceptance depends on calibration, while delayed feedback cannot evaluate edits individually. Second, automatic promotion that estimates a loss change should occur only where a comparable check can be applied at the recipient, in exact and re-runnable simulated settings, and human promotion should hold the observed outcomes and judgments. Delayed feedback supports no per-edit gate on either half. Third, the main failure risks should differ: overfitting to a suite and bloat under exact feedback, corrupted outcome channels under C2, proxy optimization under C3, and drift under C4.

These are expectations about the evidence a gate can establish, not restrictions on whether software can execute a decision. A program can merge two judged artifacts or prune a delayed proposal without estimating a loss change. Section VI makes this qualification explicit in the gate grid. In every class, sharing remains an additional empirical question: even the same exact checker must be applied to the recipient's relevant cases before broader performance is established.

VI. WHEN A LOCAL IMPROVEMENT IS SHARED

A rule can be useful in one repository and inappropriate in the next. Copying the file transfers the instruction, but not the evidence that justified it. The recipient must either share the relevant evaluation conditions or test the change on its own workload. This is the additional decision that separates local adaptation from promotion.

A. SCOPE AND RECEIVING-SIDE EVALUATION

We distinguish five nested scopes. **USER** covers one person's account or fork. **PROJECT** covers a repository, workflow, or product workspace. **STACK** is a team's curated bundle of projects. A **TENANT** is an organization or customer, and **CORE** is the scope shared by all tenants. A wider scope aggregates the losses of the deployments it contains. Promotion changes which population the edit is intended to improve, as shown in Fig. 3.

The nesting makes an important limit visible. A lower average loss at a broad scope can conceal harm to an individual

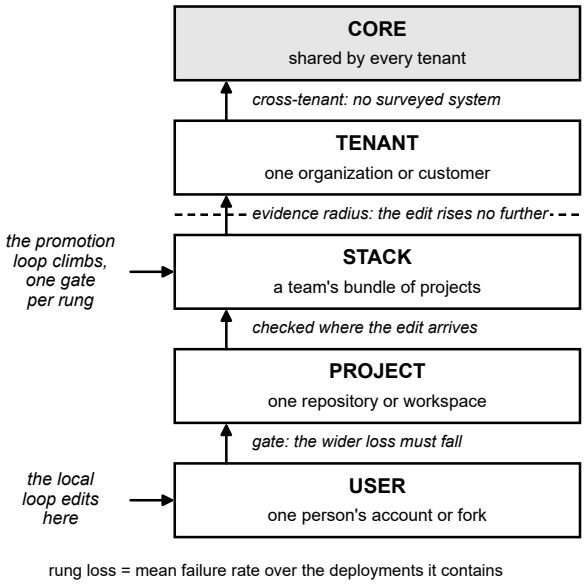


FIGURE 3. Promotion scopes, from a user account to the shared core. Wider adoption changes the population on which an edit acts. Receiving-side evaluation checks whether the improvement is supported for the recipient.

recipient. The proposed architecture in Section VIII therefore includes a held-out check at the receiving tenant. Versioning and access control govern whether an edit may be delivered and reversed; receiving-side evaluation tests whether it should be used there.

Several systems already evaluate transfers within one operator. ShareMem consolidates memories from one user for another and evaluates them in the receiving context [64]. XRepoSkill verifies a rule in at least two repositories [65]. Evo2Team promotes routing skills between teams and confirms them at the target [66]. These are concrete instances of evidence being gathered beyond the source deployment.

Federated systems place the check elsewhere. EvolveNet merges a shared harness only when the merge fixes at least as many held-out items as it breaks, but that check occurs at the aggregator [67]. FederatedSkill's server agent applies, adapts, or rejects each client's edit and records the decision [68]. Their clients are partitions or agent instances under one operator, not independently governed customer organizations. They do not instantiate a receiving-organization gate.

The location of the evaluation matters, and its coverage still matters after it is correctly located. ShareMem, XRepoSkill, and Evo2Team check the receiving context; EvolveNet checks the shared merge; FederatedSkill records a server decision. Even in Evo2Team, nearly a fifth of confirmed transfers fail on graphs not covered by confirmation. A receiving-side check provides necessary evidence for adoption, but finite evaluation does not guarantee performance on all future tasks.

B. AUTOMATIC, HUMAN, AND UNCHECKED PROMOTION

Research systems often promote by admitting a candidate to a bank or archive that later runs reuse. SkillOpt, DGM, ADAS, Voyager, and SAGE apply programmatic decisions within this shared environment [17], [52], [54], [69], [70]. AlphaEvolve and its predecessor FunSearch score candidates before archive admission [53], [71]. AlphaEvolve has produced gains used within Google, but it is a research loop within one operator, rather than a product for governed sharing between organizations. That pool is as far as any of them travels, and none has access control, audited promotion, or a gate at a receiving organization.

Not every automatic admission has the same evidentiary meaning. NanoResearch's semantic merge, AutoManual's judged rewriting and consolidation, and ShareMem's judged consolidation are programmatic decisions, but none of them estimates a candidate's reduction in patch loss. The distinction anticipated by the feedback classes is therefore essential: an automatic promotion decision can exist where a per-edit improvement estimate does not.

Production systems more often retain a person at the promotion decision. Sierra routes generated insights through human-authored expert answers [72]. In Devin, the same pull-request merge serves two roles: it supplies an observed outcome to local adaptation and records the reviewer's approval for the persistent repository [55]. Spotify's optimized prompts and plans enter production after online A/B testing and a launch decision [73]. CASCADE also combines human and agent approval for memory consolidation [57]. In each, the person at the promotion gate stands in for the exact signal production lacks, which makes a program on the local gate with a person on the promotion gate the production-viable pairing rather than a compromise.

Letta's Context Repositories take a different path. Shared-memory changes are pushed with git lineage but without an acceptance check. The record explains what was shared, while leaving whether it should have been shared unresolved. In many other systems, promotion is absent altogether. AutoAgent, for example, revises the tool and peer descriptions of one agent without propagating those revisions to another deployment [74]. Both cases receive None at the promotion gate, but for different reasons: one shares without checking; the other does not share.

C. WHAT FORMATS, REGISTRIES, AND RELEASE PLATFORMS PROVIDE

Much of the infrastructure needed for sharing already exists independently of these learning loops. Claude Agent Skills, Cursor Rules, GitHub Copilot instructions, Windsurf, Continue.dev, Zed, and Replit use git-tracked markdown artifacts attached to workspaces, with a tool layer organized around MCP-style interfaces [28], [75]. This convergence makes artifacts easier to carry between runtimes. A study of repository-level loops nevertheless finds that the runtime state of the adaptation process remains outside version control [76]. The portable file does not necessarily carry the history that produced it.

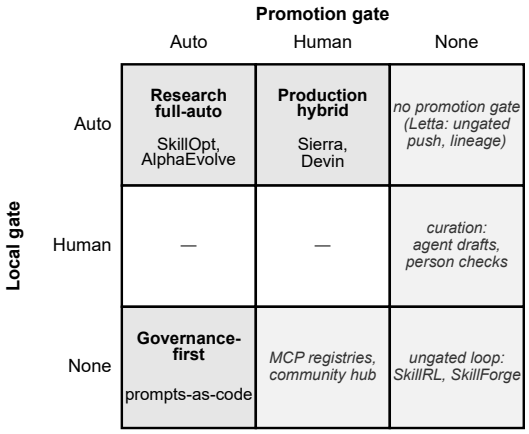


FIGURE 4. Local and promotion gates. Each axis records who accepts a change: a program, a person, or no gate. None includes both absent promotion and unchecked sharing; the text distinguishes these cases.

Public repositories provide a shared scope with human admission. In a study of five skill repositories, every substantive change was committed under a named human account and merged by a person; only one of 254 edits deprecated a skill [16]. MCP registries provide namespaces and version records, while control planes add scanning, access control, or quarantine [77]–[79]. Governance registries add lineage and rollback [80]–[82]. EDDOps gates promotion on evaluation thresholds before a human sign-off, but none of these registries generates candidate artifacts from failures.

Braintrust, Vellum, and LangSmith Hub likewise provide immutable versions, evaluation thresholds, and rollback [83]–[85]. Engineers author the candidates, and evaluations can block a regressing release. In the surveyed descriptions, session feedback does not trigger candidate generation or the release process. These platforms supply much of the machinery for controlled change without closing the failure-driven local loop.

D. THE TWO DECISIONS VIEWED TOGETHER

Fig. 4 places the local and promotion gates on a common grid. Table 4 summarizes the occupants and their signals. The grid distinguishes a system that generates and tests improvements from one that distributes human-authored artifacts under admission control, while allowing their complementary capabilities to be compared.

Among [Auto $\times$ Auto] systems whose promotion gate estimates a loss change, the operative signals are exact checks or simulated outcomes. The judged and delayed cases use pruning, rewriting, or merging rather than that estimate. [Auto $\times$ Human] contains observed outcomes and judgments, with no exact-signal system in the surveyed cell. These patterns match the feedback-based expectations once the meaning of automatic admission is made explicit.

TABLE 4. The gate grid read cell by cell: who keeps a local edit and who lets it climb, which systems sit in the cell, and the feedback signal they gate on. [Human × Auto] and [Human × Human] hold no surveyed system.

Cell	Who keeps the local edit, who lets it climb	Who is there	Signal found there
[Auto × Auto]	A program keeps it; a program lets it climb	Research systems that close both halves inside one environment or one operator	Exact and simulated outcomes; delayed and judged only under pruning or merging
[Auto × Human]	A program keeps it; a person lets it climb	Production systems with a reviewer on promotion, and CASCADE	Observed outcomes and judgments
[Auto × None]	A program keeps it; nothing lets it climb	Letta's ungated push; systems that promote nothing	As on the local gate
[Human × None]	A person keeps it; nothing lets it climb	Curation	Judged
[None × Auto]	No failure triggers a candidate; a threshold gates release	Prompts-as-code platforms and governance registries	No failure-driven loop
[None × Human]	No failure triggers a candidate; a person admits	Open standards, MCP registries, community hubs	No failure-driven loop
[None × None]	Nothing checks or promotes the edit	SkillRL and SkillForge, unchecked skill loops	Outcomes and judgments

The [Human × Auto] and [Human × Human] cells are empty in the surveyed set: no system combines a human local gate on a failure-driven loop with subsequent promotion. Human local decisions occur in curation. The bottom row includes comparison systems without a failure-driven local loop, such as prompt platforms, registries, and community hubs including Buildrix [86]. SkillRL and SkillForge occupy [None × None] because their failure-driven skill updates, new skills in SkillRL and rewrites in SkillForge, are kept unchecked.

The audit's boundary cases do not change this structure. Of six systems that promote without artifact lineage, five have sufficient evidence under the tier rule and enter already occupied cells when the audit is relaxed; Spotify is one of them. The sixth, Future AGI's prompt lifecycle, has only uncorroborated T5 evidence. No relaxation yields governed promotion across organizations. What the grid reveals is a division of capabilities: no system has both the learning of the top row and the governance of the bottom row.

VII. HOW ADAPTATION CAN UNDERMINE ITS OWN EVIDENCE

An improvement loop changes the context in which its next decision is made. New rules can conflict with old ones, a corrected memory can still be retrieved in its earlier form, and a frequently reused evaluator can become a target. These failures extend the technical-debt concerns identified for machine-learning systems by Sculley et al. [87]. The literature documents six of these failures in a measured case; a seventh, overfitting to one deployment, follows from the scope of the loss rather than from a measurement. Table 5 connects each to the control that addresses it.

A. ACCUMULATION AND CONTAMINATION

As instructions accumulate, following them becomes harder and latency increases [88]. Ratchet's finding that model-written skills need lifecycle hygiene before producing gains makes the consequence visible at the system level [34]. Conflicting or obsolete artifacts also complicate attribution: an evaluation may detect failure without revealing whether the cause is the new candidate or an older artifact. Gated retirement, pruning, and expiry preserve the conditions under which a new edit can be meaningfully assessed.

Persistent memory creates another route to lasting error. PoisonedRAG demonstrates attacks on retrieved knowledge [89], AgentPoison plants backdoors in long-term memory or knowledge bases [90], and memory control-flow attacks operate across major agent frameworks [91]. Tool outputs and connected servers can also insert instructions into the model's context [92], [93]. In CVE-2025-54136, a previously approved MCP configuration could be changed without renewed approval [94].

These failures motivate provenance, write gating, and reversible memory changes [95], [96], together with restricted tool permissions and an instruction privilege hierarchy [97]. The tool binding itself must be subject to controlled revision. Checking only the initial registration leaves later changes outside the decision that authorized it.

Rollback also needs to account for derivation. If a contaminated skill has been used to create other skills, restoring its earlier version does not remove those descendants. VaG's limited recovery after such rollback [33] supports checking contamination before commitment and tracking consequences after it. A version history is a prerequisite for this work, but it does not perform the recovery by itself.

TABLE 5. The failure surface of patchwork scaffold maintenance: what goes wrong, the piece of the loop that answers it, and the feedback-signal class in which it dominates.

Failure	What goes wrong	The loop piece that answers it	Dominant class
Scaffold bloat	Rules and skills accumulate until the model follows them worse and the gate has no clean signal	Complexity penalty Ω in the gate; pruning and expiry as first-class gated operations	C1, C3
Poisoned memory	An attacker writes an entry that later retrieval trusts	Provenance on every entry; write gating; a rollback envelope	C2
Injection via tools and MCP	A tool's output or a connected server carries instructions into context	Narrow tool permissions; a privilege hierarchy over instructions; tool bindings pass the same promotion gate as any other coordinate	C2
Evaluation fragility	The suite that gates edits is itself edited, gamed, and read non-independently	Held-out evaluations scoped within evidence radius, weighted toward recurring modes, re-read as the target drifts, checked against attribution	C3 (primary), C1
Coordination failures	Multi-agent roles blur and context is lost between agents	Orchestration topology treated as a scaffold coordinate under the same gate and rollback discipline	C2
Patch overfitting	A scaffold tuned to one deployment degrades another	Evidence-radius discipline at promotion; receiving-side held-out evaluation	C2
Plasticity loss	Old rules, polluted memory, and drifting tool versions crowd out adaptation	Expiry and rollback as explicit loop operations rather than retrospective hygiene	C4

B. WHEN THE EVALUATOR CHANGES

Consider a pitch-review assistant that consistently overrates decks claiming a large market without supporting numbers. The team revises its rubric from ''evaluate market size'' to ''require a numeric estimate and a source.'' The edit may improve reviews, but it also changes the standard used to evaluate subsequent behavior. If the same rubric judges future rubric revisions, the improvement process is modifying its own measuring instrument.

This is the reflexive-gate problem introduced with C3 feedback. The survey finds three scaffold systems reporting calibration of a judge against people: SkillForge, SkillChain, and SkillCoach [4], [19], [58]. None reports calibration for a judge that is itself being edited. The missing result is therefore more specific than an absence of calibrated judges. It concerns maintaining calibration while the criterion changes.

Weight-level reinforcement learning has encountered a related problem in extending verifiable rewards to open-ended tasks. Some methods hold the judge fixed, using checklists, pairwise preferences, references, or rubrics [98]–[101]. Others update the reward, rubric, or judge alongside the policy [102]–[106]. Among the co-evolving methods reviewed, only RLPR reports agreement with human labels, and its reward is a probability rather than a rubric. The comparison is tabulated in the audit notes (Supplementary Material S6); these weight-side methods are not included in the scaffold scores.

Even a fixed evaluator can become unreliable through repeated use. Optimizing a proxy beyond a point can reduce the underlying quality [59]. Unequal exposure to evaluation data can create apparent performance gains [107]. Evaluations of agent trajectories are also path-dependent, and deployment tasks drift while the suite remains fixed. Human criteria themselves can shift during grading [60]. Consequently, a gate needs its own held-out checks, calibration where judgment is involved, and revision history.

The failure-mode hypothesis discussed in Section IX-A adds a conditional implication: if deployment failures concentrate in recurring modes, uniformly sampled tasks may underrepresent the modes that most need correction. Evaluation should then include those modes while preserving a held-out assessment. The requirement is broader than improving a benchmark score: score alone cannot tell a harness optimizer from trial and error [14], so the evaluation must also test the claimed link between failures and edits.

C. SCOPE, COORDINATION, AND DRIFT

Some failures arise from the structure of the deployment. A study of multi-agent systems identifies fourteen failure modes, prominently including role confusion and context lost between agents [108]. Treating routing and role assignments as editable scaffold coordinates makes them subject to evaluation and rollback alongside instructions and skills.

Specialization creates a related boundary. A scaffold adapted to one deployment can degrade another because the tasks and preferred behavior differ. Receiving-side evaluation limits this risk at promotion; the evidence radius records where the improvement has been tested. This does not eliminate uncertainty outside the evaluated cases.

Finally, the scaffold can become less adaptable as old rules, polluted memories, and drifting tool versions accumulate. Long-context degradation [109] provides one related mechanism. Expiry, pruning, and rollback must remain available throughout the system's lifetime, especially when delayed feedback arrives too slowly to keep the artifacts current.

The feedback classes help organize these risks without making them exclusive to one class. Exact checks invite suite overfitting and cheap accumulation; observable outcomes depend on channels vulnerable to poisoning and injection; judged feedback is exposed to proxy optimization and moving criteria; delayed feedback makes drift difficult to correct promptly. Production systems provide individual controls, but the surface stays largely unmitigated because no surveyed system runs them together.

VIII. WHAT GOVERNED SHARING WOULD REQUIRE

The evidence so far identifies a practical integration problem. Local optimizers can propose and test changes. Release platforms can version artifacts and reverse deployments. Registries can govern admission and access. The surveyed systems do not connect these capabilities into a process that learns in one organization and shares the resulting improvement with another under equivalent controls.

We audit that missing combination through four properties:

(a) **An auto-gated local loop:** an algorithmic acceptance criterion tests a proposed artifact change.
(b) **Cross-organization promotion:** artifacts can be promoted between independently governed organizations or customers.
(c) **Versioned lineage with access control:** a promoted artifact carries its history and audit trail, with policies controlling who may access it.
(d) **Rollback:** a harmful promotion can be reversed without redeploying the whole system.

The terms *cross-tenant* and *cross-organization* refer to the same boundary here. Sharing among agents, users, or benchmark partitions within one operator does not establish property (b).

Table 6 compares the closest systems. FederatedSkill combines local adaptation with semantic skill exchange and an auditable server decision log [68]. EvolveNet evaluates shared merges and can return to the previous shared harness [67]. Both remain within one operator. Governance registries supply lineage and rollback without a local learning loop, and MCP control planes supply access control alone. Rollback exists inside one federated round and inside single deployments; no system rolls back a promotion that has crossed an organization boundary, because none makes such a promotion.

Receiving-side evaluation is an additional requirement on the promotion decision. It is kept separate from the four-property audit, which records loop, scope, and governance capabilities. A system could satisfy those capabilities while still making an unsupported decision about whether an artifact benefits its recipient.

An architecture that addresses both would contain five connected parts: per-tenant artifact pools with provenance; an aggregation or selection process that respects tenant isolation; held-out evaluation at the receiving tenant; versioned lineage and access control; and a defined reversal path for each promoted artifact. The recipient's check would reject a promotion that does not lower its own patch loss. VaG's result (Section IV) suggests that the rollback envelope would also need to track an artifact's descendants.

Federated learning offers a useful comparison, but the representation being shared is different. FedAvg combines comparable numeric updates, and its privacy and deployment problems have been extensively studied [110]–[112]. Federated prompt tuning moves the learned object to a shared prompt [113]. The nearest scaffold systems instead select, revise, or merge symbolic edits, often through a language model. Averaging is an appropriate analogy only where the artifact updates have a mergeable representation.

The extent of automation should depend on the feedback. Exact checks provide the most direct setting for testing a transferred skill, although the recipient still needs relevant test cases. Observed outcomes require evaluation on the recipient's workload because the sender's outcome is not portable evidence of success. Where quality is judged, a human promotion gate is a design feature, supported by lineage and rollback. Delayed outcomes cannot support the per-edit automatic local gate in property (a); those systems still benefit from provenance and reversibility around proposals and milestones.

Thus the four properties define a target for the settings that can support an automatic local gate. They should not be interpreted as a requirement to automate every decision in every domain. The common requirement is that the evidence attached to an edit remain meaningful where it is used, and that the system preserve the ability to inspect and reverse its adoption.

IX. HOW THE RESEARCH LINES FIT TOGETHER

The uneven architecture has a history. Different research communities began with different problems: improving prompts, learning in context, selecting agent workflows, maintaining skill libraries, or transferring experience. Each developed the part of the loop most relevant to its initial problem. Table 7 places five such lines side by side.

Prompt optimization and reflective evolution developed ways to propose edits from trajectory evidence, together with an argument for adapting artifacts on a faster schedule than weights [114], [115]. Reflective Context Learning formalizes learning in context and the need to control generalization, corresponding to the complexity penalty in the acceptance rule [116]. Harness optimization treats source-level orchestration changes as the learned object and repeatedly identifies attribution as its difficult step [9]–[12], [23], [117], [118].

The lifecycle literature starts from a different constraint: skill pools can become too large or internally inconsistent to remain useful. Its contributions include evaluated retirement, hygiene, and optimization under platform limits [13], [34]–[38], [119]. Transfer and federation ask how far a learned artifact remains useful, progressing from cross-task measurements to receiving-team confirmation and shared client updates [56], [63], [66]–[68], [120], [121].

This history helps explain why evaluation and proposal mechanisms are more developed than failure capture. Each

TABLE 6. The closest approaches to governed cross-organization scaffold optimization, scored against the four properties.

System	(a) auto-gated local loop	(b) cross-organization promotion	(c) versioned lineage with access control	(d) rollback
FederatedSkill [68]	full	partial: privacy-preserving aggregation of skill edits across heterogeneous clients that sit under one operator	partial: auditable per-modification decision log	none
EvolveNet [67]	full	partial: held-out merge gate at the aggregator, but clients are benchmark partitions, not organizations	none: no lineage, no access control	full: rollback to the previous shared harness after a failed round
SkillForge [4]	none: a calibrated judge selects the failures, nothing tests the rewrite	none: skill versions stay in the scenario deployment	partial	not documented
Governance-first registries [80]–[82]	none: no inner loop	none	partial: lineage without access control	full, with evidence
MCP control planes [77]	none: no inner loop	none	partial: access control without lineage	not reported

TABLE 7. Five research lines converging on artifact-layer descent: the problem each started from, the part of the loop it built, and what it leaves open.

Research line	Started from	The part of the loop it built	What it leaves open
Reflective evolution	Prompt optimization and continual learning	Condition 4, the candidate: proposal by reflection over trajectory evidence; the two-timescale case for editing outside the weights	A persistent artifact; a bounded step; a held-out gate
Reflective context	Learning in context as a setting of its own	Condition 5, the penalty: the learned object in context space; a generalization penalty, the Ω of the gate	A cross-deployment promotion loop
Harness optimization	Orchestration and harness engineering	Condition 3, the trace: attribution of a failure to one slot of the orchestration graph; source-level edits	Attribution across substrates; a governed promotion loop; access control; rollback
Lifecycle and multi-objective	Skill pools that bloat under platform limits	The gate on removal: gated retirement; hygiene as a precondition; a hard frontier as the shape of Ω	Closing the loop as descent; retiring a promoted artifact's descendants
Transfer and federation	How far an edit travels	The promotion loop, the check on arrival: evidence radius measured one task at a time; cross-client aggregation; receiving-side confirmation	A cross-organization gate, lineage, and rollback

research line inherits its loss from its benchmark and begins with available trajectories. Building a record of recurring failures was none of their starting problems. Attribution receives substantial attention, but usually within the substrate a particular line studies. The local loop has consequently developed from evaluation backward toward the evidence needed to choose a useful edit.

The convergence supports a shared decomposition of the system into weights and editable artifacts. GEPA, Meta-Harness, MOSS, RCL, PACE, and the harness-optimization papers all use it. A system that updates both, SIA, still treats them as separate levers [122]. The convergence does not establish that one architecture is optimal, and none of these lines documents the complete cross-organization process. It shows that different starting problems repeatedly lead to comparable mechanisms for proposing, testing, retaining, and reusing changes.

A. WHY SPECIALIZATION REMAINS OUTSIDE THE WEIGHTS

A general model and a particular deployment face different requirements. The model must work across many settings. A deployment must follow its own conventions, honor its users' preferences, and avoid failures that may be rare globally but frequent locally. The scaffold provides a place to encode those requirements with a scope narrower than the model's entire user population.

Work on reinforcement learning from human feedback reports reduced output diversity and more homogeneous preferences [123], [124]. This helps motivate the distinction between broad model behavior and customer-specific requirements. Tiwari et al. approach it from adaptation cost: in-

context changes are cheap but capacity-bounded, while in-weight adaptation is more expressive and can introduce forgetting and loss of plasticity [115], [125].

Per-customer adapters [126], distilled models, and learned routing can absorb some local requirements. They also introduce separate training, evaluation, and serving lifecycles. In practice they are rare. Adaptation lands in persistent artifacts, the one substrate that is cheap, inspectable, reversible, scoped to one customer, and updatable daily.

Two earlier papers in this series offer a hypothesis for why targeted artifact changes may be effective. *Beyond Exponential Decay* argues that long-output reliability depends on a sparse set of key decisions rather than a uniform error probability at every token [127]. Its evidence includes long-range dependence measured on one natural-text corpus, where fewer than a tenth of tokens depend on distant context, and the accuracy recovered through majority voting on reasoning benchmarks. Its argument is limited when the number of consequential decisions grows with the task, as in long-horizon agentic work.

The Architecture of Errors considers a bounded deployment with recurring tasks, tools, and evaluator expectations [128]. It argues that failures there fall into a catalogue of modes whose size grows slowly with observed experience. Under its assumptions, the intervention budget grows polylogarithmically with sequence length and becomes constant once the catalogue saturates. Its negative result is equally relevant: a finite catalogue cannot cover an unbounded domain.

If these hypotheses generalize, a deployment may be improved through a manageable set of targeted interventions. Persistent rules, memories, skills, and tool changes provide an inspectable representation for those interventions. On that hypothesis the gated and scoped artifact layer is not a workaround for an inadequate model but the architecture the error structure calls for: a frontier model that generalizes, a local scaffold that specializes, and reliability that comes from governing the specialization. The survey does not test the hypotheses; its findings about acceptance gates, feedback, and promotion follow from the corpus independently of them.

B. RELATION TO NEIGHBORING SURVEYS

Existing surveys organize the field around agent evolution, persistent state, context engineering, skills, memory security, and harness design. This survey follows a different unit of analysis: the decision that accepts a persistent change, the feedback supporting that decision, and the scope in which the change is subsequently used. Table 8 records the distinction for each neighboring survey.

The closest governance precedents are Ding et al.'s survey of persistent-state agents and Lin et al.'s memory-security survey [132], [133]. The former finds greater attention to accumulating and retrieving state than to governing, recovering, or relinquishing it. The latter treats scoping and rollback as security primitives. Neither asks who gates an edit, and neither covers promotion across organizations.

Prompt optimization supplies another important lineage: OPRO, EvoPrompt, Promptbreeder, DSPy, and TextGrad optimize a prompt or program once against development evidence and are not deployed loops [138]–[142]. ACE persists an evolving playbook but never promotes it [143]. Agent symbolic learning treats prompts, tools, and pipelines as symbolic parameters [144]; STOP improves its own scaffolding program [145]; and SICA edits its harness to improve benchmark performance [146]. These works establish many of the underlying mechanisms.

The contribution here is their joint reading at deployment scope: whether an artifact persists, whether a local acceptance decision is made, which signal supports it, and what authorizes use elsewhere. The 256-system corpus, two-loop classification, feedback classes, and governance audit make that comparison explicit. An extended discussion separates these contributions from their precedents (Supplementary Material S7).

X. OPEN PROBLEMS AND CONCLUSION

The remaining problems become more specific when expressed in terms of the decisions a deployment must make.

Attributing a failure to an editable cause. An exact check can establish that an output failed without identifying the instruction, memory, skill, or tool binding responsible. No surveyed method solves this across all substrates. Slot attribution [12], intervention-based attribution [22], and contrastive multi-agent attribution [10] provide narrower starting points. Observed outcomes make the problem harder because their causes are more confounded.

Establishing improvement at the recipient. A transferred artifact must be evaluated against the receiving deployment's tasks and measure. Existing receiving-side checks show that this is feasible in selected settings, while Evo2Team's failures on unseen graphs show its coverage limits. Federated Agent Optimization proposes a receiving-client gate but does not implement it [147]. Connecting such checks to provenance, access control, and reversal remains an architectural problem.

Maintaining calibration while the criterion changes. For judged tasks, the open question is how to keep a judge aligned with people while revising the rubric it applies. No surveyed scaffold system reports this measurement. The neighboring reward literature also lacks a reported human-agreement result for a co-evolving rubric or judge, with RLPR's probability-based reward a distinct case. A useful solution must evaluate the moving criterion as well as the outputs it scores.

Evaluating improvement with delayed outcomes. When many edits occur before a meaningful result arrives, per-edit acceptance is unavailable. Proxy milestones and proposal pruning may still be useful, but downstream gains alone do not establish successful attribution. Harness optimizers should therefore be assessed against their claims about why particular edits help, as well as their final scores [14].

Connecting local learning to governed sharing. No surveyed system combines automatic local acceptance, promo-

TABLE 8. Neighboring surveys and where this one departs from each. This survey classifies by loop closure, gate type, signal class, and governance, and is the only one to cover promotion across organizations.

Survey	Scope	How this survey differs
Gao et al. [129]	self-evolving agents	organizes by what, when, how, and where an agent evolves; the gate that accepts an edit, and the edit's scope, are mentioned but not classified
Fang et al. [130]	self-evolving agents	organizes by inputs, agent system, environment, and optimizer; loop closure appears in passing, gates and promotion do not
Zhou et al. [131]	self-evolving coding agents (108 methods)	organizes by evolution target, timing, and code signal; closest on reversibility, with no gate or promotion axis
Ding et al. [132]	persistent-state agents (435 works)	organizes by authority, scope, mutability, provenance, and recoverability of state; governance is an axis, the gate on an edit is not
Lin et al. [133]	memory security (~90 works)	organizes by lifecycle phase and security objective; principal scoping and rollback are axes, feedback and promotion are not
Guo et al. [134]	agent system and harness design	organizes by six runtime responsibilities; touches all four questions without classifying by any
Xia et al. [135]	skill-mediated agents (8 systems)	organizes by four responsibility layers and ten patterns; loop closure and governance appear as patterns, not as axes
Ding et al. [136]	skill evaluation and evolution	organizes by four evolution paradigms and six benchmark classes; evaluation is the subject, the gate it feeds is not
Mei et al. [137]	context engineering (1,411 papers)	organizes by components and system implementations; adaptation loops lie outside its scope

tion across organizations, versioned lineage with access control, and rollback. The components exist in different systems. Their integration must preserve tenant isolation, evaluate the recipient, and account for artifacts derived from a change that is later withdrawn. Where feedback cannot support automatic acceptance, the design must retain the corresponding human decision or delayed evaluation.

The survey's central observation is that persistent changes outside the weights have become an important form of adaptation, while the evidence supporting those changes remains fragmented. Many systems can test a local candidate. Fewer can reliably explain which recurring failure it addresses, and the machinery for transferring the resulting improvement to another organization has not been assembled in the public systems examined here.

The frontier model generalizes. The local scaffold specializes. Reliability comes from governing that specialization: knowing what each change improved, where that evidence applies, and how the change can be reversed. That discipline must extend to the evaluations themselves when they become part of the adapting system.

XI. APPENDIX: FORMAL STATEMENT OF ARTIFACT-LAYER DESCENT

This appendix states the setup behind Section II, a one-step descent inequality under a noisy gate, an informal convergence proposition, the optimization-analogy correspondence, and four worked cases. No new theorem is claimed; the proposition is the ordinary target of local stochastic optimization under noisy estimators, restated for a discrete, versioned parameter.

A. SETUP

Let the frontier model have fixed parameters θ_M and let the scaffold be a structured object $\theta_s \in \Theta_s = \Theta_1 \times \cdots \times \Theta_6$ over the six substrate coordinates of Section III; Θ_s is discrete, symbolic, and versioned. For a deployment D, $L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]$. A candidate edit $z_t \sim Q_t(z \mid H_t, \theta_s^t)$ is drawn from a proposal process conditioned on the accumulated history H_t of failures, corrections, rejected outputs, evaluation regressions, and prior accepted edits. Q_t is not uniform: for a mode m that recurs with frequency f_m and carries per-occurrence loss s_m, its expected contribution to the failure rate is $f_m s_m$, so high-frequency or high-severity modes are proposed first. The finite directional difference is $\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s)$; the gate sees only a noisy estimate $\widehat{\Delta}_z L_D$. The update is

$$\theta_s^{t+1} = \theta_s^t \oplus z_t \text{ if } G_t(z_t) = 1, \quad \theta_s^{t+1} = \theta_s^t \text{ otherwise.}$$

B. DESCENT INEQUALITY

Assume (A1) positive descent bias on accepted edits, $\mathbb{E}[\widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t] \le -\gamma_t - \tau_t$ for some $\gamma_t > 0$, and (A2) bounded estimator bias on accepted edits, $|\mathbb{E}[\Delta_{z_t} L_D - \widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t]| \le \varepsilon_t$. With $p_t = \Pr(G_t = 1 \mid \theta_s^t)$,

$$\mathbb{E}[L_D(\theta_s^{t+1}) \mid \theta_s^t] \le L_D(\theta_s^t) - p_t(\gamma_t + \tau_t) + p_t\,\varepsilon_t.$$

Per-step expected improvement is the gate's margin minus its estimator bias, weighted by acceptance probability. In the feedback-signal classes of Section V, ε_t is near zero for the check itself in C1, with the sampling error of a finite suite over the patch remaining, bounded by batching for an observable outcome (C2), equal to the judge's bias and bounded only by calibration for a judged signal (C3), and unavailable per edit for a delayed signal (C4).

TABLE 9. Optimization concepts and their scaffold analogues.

Optimization concept	Scaffold analogue
Parameter vector	Instructions, skills, memories, tools, routing graphs, and governance rules, evaluation gates among them
Loss	Failure, user correction, evaluation regression, retrieval miss, tool error, schema violation
Gradient estimate	Credit assignment identifying the coordinate responsible for a failure
Step	Accepted scaffold edit; step size is edit magnitude under $\oplus$
Aggregated update	Several recurring failures combined before an update, by averaging or by agreement across k proposals
Update radius	Promotion scope: which loss is descended (USER, PROJECT, STACK, TENANT, CORE)
Validation loss	Held-out patch evaluation before merge
Regularization	Complexity penalty $\Omega(z)$, bloat control, cross-patch regression checks
Overfitting	Scaffold improves on this deployment and degrades elsewhere
Rollback	Reverting a harmful accepted update

C. CONVERGENCE (INFORMAL)

Add (A3) proposal coverage with acceptance: at every non-gate-stable scaffold, Q_t places probability at least $q > 0$ on an admissible edit with $\mathbb{E}[\Delta_z L_D] \leq -\delta$, and the gate accepts such an edit with probability at least $q' > 0$, so that $p_t(\gamma_t + \tau_t) \geq qq'\delta$ there; (A4) effective compactness of the reachable orbit; and (A5) summable bias, $\sum_t p_t \varepsilon_t < \infty$; and let $0 \leq L_D \leq B$. The one-step inequality is then of the Robbins–Siegmund form, a non-negative process that decreases in expectation up to a summable perturbation $p_t\varepsilon_t$, so $L_D(\theta_s^t)$ converges almost surely and $\sum_t p_t(\gamma_t + \tau_t) < \infty$ almost surely [148]. The finite sum carries the content: a bounded loss cannot pay for infinitely many accepted edits with a positive expected margin, so under (A3) the process visits non-gate-stable scaffolds only finitely often, and with (A4) the limit set of $\{\theta_s^t\}$ is gate-stable: no admissible edit clears the gate with negative expected directional loss in the limit. This is convergence of the loss and stability of the gate decision, not convergence to an optimum; a globally suboptimal gate-stable scaffold is consistent with the result, because Q_t may never propose the improving edit or $\widehat{\Delta}$ may be biased to reject it. In a non-stationary deployment ($D = D_t$) the same mechanism tracks a moving optimum rather than converging once, which is why recency weighting, pruning, expiry, and rollback are the scaffold-level analogues of continual-learning machinery rather than optional hygiene.

D. CORRESPONDENCE

Table 9 pairs each optimization concept with its scaffold analogue. Credit assignment here is attribution, not back-propagation. Scaffold systems do not differentiate through instruction files. They trace an output error to an upstream coordinate, manually or with a judge or verifier. The descent argument needs only that this attribution succeeds with non-zero probability per failure, which is the credit-assignment condition of Section II.

E. WORKED CASES

Retrieval. Failures recur because retrieved context omits the decisive clause. Attribution traces the error to the retrieval coordinate: chunking, query rewriting, metadata filter, embedding model, reranker, or context-packing rule. The candidate edit tightens chunk boundaries or adds a metadata constraint, and the gate checks that recall and answer accuracy improve on the patch evaluation without degrading adjacent cases. Signal class C1 or C2.

Pitch review. The system repeatedly overrates decks that claim a large market without numbers, and the failure cluster points at an under-specified rubric coordinate. The edit changes "evaluate market size" into "require a numeric estimate and a source; if absent, cap the score"; the gate runs the revised rubric against held-out decks with human scores. What the edit changed was the gate that will judge the next edit: the reflexive gate of Section V, class C3.

Tool use. Calls fail because optional parameters are under-specified. The responsible coordinate may be the tool schema, the call policy, or the clarification rule. The edit adds required fields or a pre-call uncertainty check; the gate checks that tool-call failures fall without raising refusals or latency. Class C2.

Memory. A stale customer preference is repeatedly retrieved and causes bad decisions. The responsible coordinate is a memory entry together with its provenance, priority, and expiry policy. The edit corrects, expires, or version-tags the entry; the gate checks that future behavior improves without erasing useful history. Class C2, with the correction itself observable only when a later outcome reveals it.

XII. APPENDIX: GATE FAMILIES AND FEEDBACK-SIGNAL CLASSES

Section II collapses every production gate into one per-edit test G_t on an estimate of the loss change. Surveyed systems instantiate that test in several distinct ways, and some of them are better read as proposal pruning that precedes the per-edit gate than as estimators of $\widehat{\Delta}L_D$. Table 10 lists the families, the slot each occupies, representative systems, and the feedback-signal class (Section V) each family serves.

The first five families populate G_t directly and, under the assumptions of Appendix XI, inherit its descent guarantee. Semantic merge and archive selection are not estimators of

TABLE 10. Gate families, their slot in the formalism, and the feedback-signal class each serves.

Gate family	What it estimates	Slot in Section II	Signal class	Representative systems
Surrogate verifier	Pass or fail from tests, a symbolic checker, or an exact match	Per-edit gate G_t	C1	DGM (sandboxed compile-and-edit check; score weights selection, not retention), SkillOpt (held-out exact match), Voyager (game-state check: C2)
Evaluation cascade	$\widehat{\Delta}L_D$ on a held-out task suite, cheap-to-expensive evaluators	Per-edit gate G_t	C1 when the suite is exact; C2 when it is a task suite	AlphaEvolve, Braintrust, Vellum, LangSmith Hub
RL threshold	Reward improvement against a deployment threshold	Per-edit gate G_t	C2	SAGE, PACE
LLM judge with calibration	Pairwise or rubric score, calibrated against human agreement	Per-edit gate G_t	C3	SkillChain (judge stage, before its human reflection gate)
Reviewer judgment	Human credit assignment by pull-request review, expert sign-off, or a user's acceptance	Per-edit gate G_t, human-instantiated	C3 (and C2 when the reviewer has outcome data)	Sierra Agent OS 2.0, Anthropic skills repository, CASCADE (a person and an agent approve consolidation into shared memory)
Semantic merge	Whether a candidate duplicates or conflicts with existing artifacts	Proposal pruning before G_t	C4 (also a pre-filter in C2, C3)	NanoResearch orchestrator, FederatedSkill server agent, EvolveR (admission: cosine pre-filter plus LLM equivalence judgment on each distilled principle; retention: periodic pruning on per-principle success rate)
Archive selection	Whether a candidate improves on, or diversifies, an archive of already-scored programs	Proposal pruning before G_t	Any; inherits the class of the evaluator that scored the archive (C1 in AlphaEvolve)	AlphaEvolve (MAP-Elites-style database), ADAS
Multi-sample agreement	Whether k independent proposals or evaluations agree; false-accept rate decays as p^k rather than variance as $1/k$	Aggregation filter on the estimate	C2, C3 noise absorption	VaG (three heterogeneous critics must all pass); SkillOpt-Lite (consensus filter across rollouts)

$\widehat{\Delta}L_D$; they constrain the proposal distribution, and the descent argument applies to the composition of pruning and gate. Multi-sample agreement is listed because it is the correct analogue of "wait for k pieces of evidence before accepting an edit", a different operation from mini-batch averaging and one that should not be conflated with it. Read down the signal-class column. The only families available to a judged (C3) deployment are the two whose estimator is a judge. A delayed (C4) deployment has no family that estimates the loss change per edit at all: what it has is semantic merge before the gate and whatever proxy milestone stands at the horizon. That is the structural content of Section V.

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) to help search the literature, check citations, and edit prose; all claims, scores, and classifications were reviewed and approved by the authors. Earlier versions of this argument appeared as an arXiv preprint and as a non-archival workshop submission; this article extends them with the feedback-signal classification of Section V and a corpus scored through October 2026.

REFERENCES

[1] M. Zaharia, O. Khattab, L. Chen, J. Q. Davis, H. Miller, C. Potts, J. Zou, M. Carbin, J. Frankle, N. Rao, and A. Ghodsi, "The shift from models to compound AI systems," Berkeley Artificial Intelligence Research (BAIR) Blog, https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, 2024.

[2] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*, 2023, arXiv:2306.05685.

[3] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents (CoALA)," *Transactions on Machine Learning Research (TMLR)*, 2023.

[4] X. Liu *et al.*, "SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support," *arXiv preprint*, 2026, accepted at ACM SIGIR 2026 Industry Track.

[5] P. Xia *et al.*, "SkillRL: Evolving agents via recursive skill-augmented reinforcement learning," *arXiv preprint*, 2026.
[6] M. Helwig, "Memory as infrastructure: Reliability engineering for persistent agent memory in months-long LLM-assisted development," *arXiv preprint arXiv:2609.05510*, Aug. 2026.
[7] A. Miyai, K. Aizawa, and T. Yamasaki, "Task-coevolve: Efficient harness optimization via adaptive validation task selection," *arXiv preprint arXiv:2608.20169*, 2026.
[8] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang, "Autosaddler: Automatic harness optimization with durable updates from agent execution traces," *arXiv preprint arXiv:2608.23041*, 2026.
[9] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," *arXiv preprint*, 2026.
[10] T. Zehle, "CANTANTE: Optimizing agentic systems via contrastive credit attribution," *arXiv preprint*, 2026.
[11] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn, "Meta-Harness: End-to-end optimization of model harnesses," *arXiv preprint*, 2026.
[12] M. Nguyen, W. C. Tan, N. A. Hassan, A. Raman, L. H. Lim, and A. F. Razak, "Where does harness-optimization value live? Localized gains and the budget-splitting trap in self-evolving LLM agents," *arXiv preprint arXiv:2609.02889*, 2026.
[13] H. Liu, H. Yang, T. Jiang, B. Tang, F. Xiong, Y. Luo, and Z. Li, "SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution," *arXiv preprint arXiv:2605.18401*, 2026.
[14] K. T.-i. Ong, M. Kang, D. Choi, J. Cho, S. Kim, S. Lim, G. Jang, M. Oh, B. Jeong, S. Kim, T. Kwon, and J. Yeo, "Towards direct evaluation of harness optimizers via priority ranking," *arXiv preprint*, 2026.
[15] Y. Shen, B. Li, and X. Zhang, "SkillOpt-Lite: Better and faster agent self-evolution via one line of vibe," *arXiv preprint arXiv:2607.03451*, 2026.
[16] C. Shen and E. Hruschka, "Who maintains agent skills? A longitudinal study of human-governed, AI-assisted skill maintenance," *arXiv preprint arXiv:2609.05677*, 2026.
[17] J. Wang *et al.*, "Reinforcement learning for self-improving agent with skill library," *arXiv preprint*, 2025.
[18] X. Wu *et al.*, "Co-evolving LLM decision and skill bank agents for long-horizon tasks," *arXiv preprint*, 2026.
[19] Y. Hu, M. Xu, H. Guo, Y. Song, X. Zhu, and B. Zheng, "Skillchain: Closing the loop on skill evolution for image-based e-commerce ai assistants," *arXiv preprint arXiv:2606.12984*, 2026.
[20] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "Language agents as optimizable graphs," in *Forty-first International Conference on Machine Learning (ICML 2024)*, 2024, arXiv:2402.16823.
[21] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," *arXiv preprint arXiv:2410.10762*, 2024.
[22] J. Chu, J. Seo, J. Cho, J. Na, Y. Xiong, Y. Kim, and H. J. Kim, "Agentgrad: Intervention-guided prompt optimization for multi agent systems," *arXiv preprint arXiv:2609.08572*, 2026.
[23] M. Chen, C. Lv, G. Zhang, H. Chang, and S. Zhou, "Harnessforge: Joint harness and policy evolution for adaptive agent systems," *arXiv preprint arXiv:2606.01779*, 2026.
[24] S. Zhao, R. Kong, Z. Zhang, L. Shen, X. He, X. Zhang, and Q. Zhang, "Topological coherence for self-evolving multi-agent systems," *arXiv preprint arXiv:2609.37953*, 2026.
[25] Y. Cheng, C. Liu, H. Yu, H. Tan, T. Wu, W. Jin, J. Ran, K. Wang, X. Li, H. Bai, G. Chesi, and N. Wong, "Optimas: Automatically optimize multi-agent system," *arXiv preprint arXiv:2608.21918*, 2026.
[26] AGENTS.md, "AGENTS.md: A cross-vendor open standard for agent instructions," 2025. [Online]. Available: https://agents.md
[27] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic much? adoption of coding agents on github," *arXiv preprint arXiv:2601.18341*, 2026.
[28] M. Galster, S. Mohsenimofidi, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "Harness engineering for agentic ai coding tools: An exploratory study," *arXiv preprint arXiv:2602.14690*, 2026.
[29] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating agents.md: Are repository-level context files helpful for coding agents?" *arXiv preprint arXiv:2602.11988*, 2026.
[30] J. L. Lulla *et al.*, "On the impact of AGENTS.md files on the efficiency of AI coding agents," *arXiv preprint*, 2026.
[31] T. Q. Nguyen, S. Desai, R. H. Anwar, F. Shaik, V. Suryanarayanan, and V. Chowdhary, "VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents," *arXiv preprint arXiv:2506.02539*, Jun. 2025.
[32] Y. Wang, L. Xia, R. Liu, Z. Zhang, H. Wu, Y. Yang, J. He, Y. Guo, and K. Lei, "Sage: A statistical acceptance gate for self-evolving agents," *arXiv preprint arXiv:2609.36043*, 2026.
[33] L. Shang, M. Xu, Y. Sun, T. Xia, L. Hu, L. Xu, and N. Zheng, "When self-evolution backfires: Pre-commit gating against skill contamination in llm agents," *arXiv preprint arXiv:2608.05810*, 2026.
[34] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Ratchet: A minimal hygiene recipe for self-evolving LLM agents," *arXiv preprint*, 2026.
[35] ——, "Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries," *arXiv preprint arXiv:2605.19576*, 2026.
[36] Y. Wei, Z. Huang, S. Lu, J. Qian, Q. Wang, C. Wu, and L. He, "Skillsmith: Co-evolving skills and tools for self-improving agent systems," *arXiv preprint arXiv:2606.01314*, 2026.
[37] H. Pu, X. Song, and L. Zhao, "Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems," *arXiv preprint arXiv:2605.13716*, 2026.
[38] Y. Wang, Y. Zhou, Y. Liang, C. Zhang, F. Liu, J. Zhou, and H. Yao, "Not all skills help: Measuring and repairing agent knowledge," *arXiv preprint arXiv:2606.15390*, 2026.
[39] Letta, "MemFS; memory & dreaming; introducing context repositories: Git-based memory for coding agents," https://docs.letta.com/letta-code/memfs, https://docs.letta.com/letta-agent/memory, https://www.letta.com/blog/context-repositories/, 2026.
[40] Mem0, "Mem0 platform: Memory history, feedback, and changelog," https://docs.mem0.ai/api-reference/memory/history-memory, https://docs.mem0.ai/api-reference/memory/feedback, https://docs.mem0.ai/changelog, 2026.
[41] Zep, "Zep documentation: Facts; february 2026 deprecation wave; changelog," https://help.getzep.com/facts, https://help.getzep.com/february-2026-deprecation-wave, https://help.getzep.com/changelog/2026/9/24, 2026.
[42] Y. T. Shen, K. Toyoda, and A. Leung, "Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems," *arXiv preprint arXiv:2609.08258*, Sep. 2026.
[43] Z. Wang, "TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory," *arXiv preprint arXiv:2606.06240*, Jun. 2026.
[44] J. Xu, Y. Xiao, W. Shao, H. Liu, and X. Li, "Memory provenance laundering in LLM agents: A non-amplification firewall for persistent memory," *arXiv preprint arXiv:2607.29167*, Jul. 2026.
[45] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed shared memory for multi-agent LLM systems," *arXiv preprint arXiv:2606.24535*, Jun. 2026.
[46] X. Sun, Q. Wang, H. Li, and L. Chen, "Verifiable memory: Learning unified memory management with local and global verifiers for large language model agents," *arXiv preprint arXiv:2608.03137*, Aug. 2026.
[47] Z. Yu, Y. Wu, Z. Yin, K. Chen, Z. Zhao, M. Wang, S. Yan, and L. Yang, "Recursive experiential–working memory evolution for long-horizon agent harnesses," *arXiv preprint arXiv:2608.24876*, Aug. 2026.
[48] H. Wang, G. Dong, H. Liang, Z. Zhang, J. Luo, C. Liu, C. Xue, and H. Tang, "MemGuard: Persisting verifier signals for LLM-agent memory governance," *arXiv preprint arXiv:2608.21867*, Aug. 2026.
[49] D. F. Cuadros, A.-A. Maiga, H. Meskhidze, and A. Curtis-Trudel, "Governed collaborative memory as artificial selection in LLM-based multi-agent systems," *arXiv preprint*, 2026.
[50] H. Gao, H. Chen, C. Wang, S. Guo, L. Pang, Z. Liu, H. Shen, and X. Cheng, "Skillaudit: Ground-truth-free skill evolution via paired trajectory auditing," *arXiv preprint arXiv:2606.14239*, 2026.
[51] T. Sah, D. Sah, H. Jain, and T. Sah, "Evoundo: Recoverability-constrained self-evolution for llm agent harnesses," *arXiv preprint arXiv:2608.28363*, 2026.
[52] Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, K. Qiu, Y. Yang, D. Chen, X. Yang, and C. Luo, "SkillOpt: Executive strategy for self-evolving agent skills," arXiv preprint arXiv:2605.23904, 2026. [Online]. Available: https://microsoft.github.io/SkillOpt

[53] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv preprint arXiv:2506.13131, 2025.
[54] J. Zhang *et al.*, "Darwin Gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint*, 2025.
[55] Cognition, "Devin's 2025 performance review: Learnings from 18 months of agents at work," https://cognition.com/blog/devin-annual-performance-review-2025, 2025.
[56] C. Ling, P. Chen, A. Guan, J. Qu, S. A. Akbar, M. Gopinathan, and E. Cornejo, "PACE: Two-timescale self-evolution for small language model agents," *arXiv preprint*, 2026.
[57] X. Huang *et al.*, "CASCADE: Cumulative agentic skill creation through autonomous development and evolution," *arXiv preprint*, 2025.
[58] J. Zhu, K. Mao, Y. Guo, D. He, S. Xu, S. Gu, and Y. Yue, "Skillcoach: Self-evolving rubrics for evaluating and enhancing agentic skill-use," *arXiv preprint arXiv:2607.01874*, 2026.
[59] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.
[60] S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences," *arXiv preprint arXiv:2404.12272*, 2024.
[61] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," *arXiv preprint arXiv:2408.06292*, 2024.
[62] J. Xu *et al.*, "NanoResearch: Co-evolving skills, memory, and policy for personalized research automation," *arXiv preprint*, 2026.
[63] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, "CORAL: Towards autonomous multi-agent evolution for open-ended discovery," *arXiv preprint*, 2026.
[64] J. Hu, H. Zhao, Q. Jia, D. Chen, T. Zhao, S. Duan, and G. Liu, "Learning from others, acting for you: Cross-user memory sharing for llm agents," *arXiv preprint arXiv:2609.32511*, 2026.
[65] Y. Guo, H. Zhou, J. Zhang, Y. Zhang, Y. Liu, Q. Chen, Q. Lin, H. Dong, J. M. Zhang, and Z. Chen, "Xreposkill: Learning transferable skills for software engineering agents," *arXiv preprint arXiv:2609.36807*, 2026.
[66] R. Wang and J. Cui, "Evo2team: When do evolved skills transfer? from selection to deployment," *arXiv preprint arXiv:2609.34135*, 2026.
[67] J. Nie, Y. Zhang, Q. Cai, Y.-m. Cheung, X. Tian, and B. Han, "Evolvenet: Collaborative harness evolution for agent self-improvement," *arXiv preprint arXiv:2608.04968*, 2026.
[68] J. Yang, G. Yao, Y. Zhang, R. R. Kompella, G. Liu, and S. Chang, "FederatedSkill: Federated learning for agentic skill evolution," *arXiv preprint arXiv:2606.03143*, 2026.
[69] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems (ADAS)," *arXiv preprint*, 2024.
[70] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint*, 2023.
[71] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.
[72] Sierra, "Agent OS 2.0: From answers to memory and action," 2025. [Online]. Available: https://sierra.ai/blog/agent-os-2-0
[73] E. Palumbo, A. Tamborrino, V. Ode, B. Lacker, A. Casas Escoda, J. Hopple, M. Better, J. Leoni, H. Galvão, H. Bouchard, M. Lalmas, J. L. Redondo García, A. Abebe, A. Clifton, A. Blomberg, H. Lindström, D. Doro, and C. Doig Cardet, "Bootstrapping conversational recommendation agents at spotify: Synthetic data generation and self-improvement loops," *arXiv preprint arXiv:2609.30297*, 2026.
[74] X. Wang *et al.*, "AutoAgent: Evolving cognition and elastic memory orchestration for adaptive agents," *arXiv preprint*, 2026.
[75] P. Barbaste, T. Darrigol, G. Vu, and T. Wiltberger, "Harness engineering: Anatomy, architecture, and evolution of coding agents – a source-code study of eleven systems," *arXiv preprint arXiv:2609.00006*, 2026.
[76] J. L. Lulla, V. Nersesyan, S. Mohsenimofidi, C. Treude, and S. Baltes, "Loop engineering: Building blocks, adoption, and impact," *arXiv preprint arXiv:2608.21884*, 2026.
[77] agentic-community, "Mcp gateway & registry," https://github.com/agentic-community/mcp-gateway-registry, 2026.
[78] Microsoft, "Inventory and discover mcp servers in your api center," https://learn.microsoft.com/en-us/azure/api-center/register-discover-mcp-server, 2026.
[79] TrueFoundry, "Mcp gateway authentication and security," https://www.truefoundry.com/docs/ai-gateway/mcp-gateway-auth-security, 2026.
[80] R. Kang and V. Wang, "Registry-governed agent lifecycle: Completing EDDOps with evaluation-driven registration, promotion, and retirement on AWS AgentCore," *arXiv preprint arXiv:2607.00345*, Jul. 2026.
[81] A. Quessada-Vial, "Agentic configuration management (ACM): A reference configuration model for governed agentic systems," *arXiv preprint arXiv:2608.11166*, Aug. 2026.
[82] X. Qin, S. Luan, J. See, Z. Boukhers, C. Yang, and Z. Li, "Governed capability evolution: Lifecycle-time compatibility checking and rollback for AI-component-based systems, with a proof-of-concept evaluation on embodied agents," *arXiv preprint arXiv:2604.08059*, Apr. 2026.
[83] Braintrust, "Use prompts in code," Braintrust documentation, https://www.braintrust.dev/docs/guides/prompts, 2026, accessed 2026-10-03.
[84] Vellum, "Release tags," Vellum documentation, https://docs.vellum.ai/product/deployments/release-tags, 2026, accessed 2026-10-03.
[85] LangChain, "Prompt engineering concepts," LangSmith documentation, https://docs.langchain.com/langsmith/prompt-engineering-concepts, 2026, accessed 2026-10-03.
[86] Z. Jiang and B. Dong, "Buildrix: An open platform for sharing and benchmarking agentic ai skills in building engineering," *arXiv preprint arXiv:2606.25139*, 2026.
[87] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, 2015.
[88] D. Jaroslawicz *et al.*, "How many instructions can LLMs follow at once?" *arXiv preprint*, 2025.
[89] W. Zou *et al.*, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," *arXiv preprint*, 2024, accepted at USENIX Security 2025.
[90] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," *arXiv preprint arXiv:2407.12784*, 2024.
[91] Z. Xu *et al.*, "From storage to steering: Memory control flow attacks on LLM agents," *arXiv preprint*, 2026.
[92] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
[93] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," *arXiv preprint arXiv:2406.13352*, 2024.
[94] Check Point Research, "MCPoison: Cursor IDE's MCP vulnerability (CVE-2025-54136)," https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/, 2025.
[95] Y. Su, W. Xu, C. Zuo, and E. Bertino, "ChronoMem: Version control and semantic rollback for large language model agent memory," *arXiv preprint arXiv:2607.27773*, 2026.
[96] C. Yu, Y. Wang, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai, "From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents," *arXiv preprint arXiv:2608.10502*, 2026.
[97] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.
[98] A. Gunjal, A. Wang, E. Lau, V. Nath, Y. He, B. Liu, and S. Hendryx, "Rubrics as rewards: Reinforcement learning beyond verifiable domains," *arXiv preprint arXiv:2507.17746*, 2025.
[99] R. Jia, Y. Yang, Y. Gai, K. Luo, S. Huang, J. Lin, X. Jiang, and G. Jiang, "Writing-zero: Bridge the gap between non-verifiable tasks and verifiable rewards," *arXiv preprint arXiv:2506.00103*, 2025.
[100] K. Shi, Y. Liu, P. Wang, A. R. Fabbri, S. Joty, and A. Cohan, "References improve LLM alignment in non-verifiable domains," *arXiv preprint arXiv:2602.16802*, Feb. 2026.
[101] X. Jin, J. Zhang, T. Cao, and B. Yang, "Beyond score prediction: LLM-based essay scoring and feedback generation via reinforcement learning with rubric rewards," *arXiv preprint arXiv:2607.19219*, Jul. 2026.
[102] T. Yu, B. Ji, S. Wang, S. Yao, Z. Wang, G. Cui, L. Yuan, N. Ding, Y. Yao, Z. Liu, M. Sun, and T.-S. Chua, "RLPR: Extrapolating RLVR to general domains without verifiers," *arXiv preprint arXiv:2506.18254*, Jun. 2025.
[103] R. Xu, T. Liu, Z. Dong, T. Yu, I. Hong, C. Yang, L. Zhang, T. Zhao, and H. Wang, "Alternating reinforcement learning for rubric-based re-

ward modeling in non-verifiable LLM post-training," *arXiv preprint arXiv:2602.01511*, 2026.

[104] H. Ding, B. Huang, Y. Fang, W. Liao, Z. Li, J. Zhang, Z. Wu, J. Zhao, and Y. Wang, "EvoRubrics: Dynamic rubrics as rewards via adversarial co-evolution for LLM reinforcement learning," *arXiv preprint arXiv:2606.23038*, Jun. 2026.

[105] H. Zhu, C. Cai, Y. Song, X. Chen, S. Han, and Y. Guo, "Self-evolving deep research via joint generation and evaluation," *arXiv preprint arXiv:2606.04507*, Jun. 2026.

[106] G. Li, B. Dalvi Mishra, Z. Wang, J. Yan, Y. Chen, C.-L. Li, L. T. Le, R. Han, G. Lee, H. Tong, C.-Y. Lee, and T. Pfister, "RubricEM: Meta-RL with rubric-guided policy decomposition beyond verifiable rewards," *arXiv preprint arXiv:2605.10899*, May 2026.

[107] S. Singh *et al.*, "The leaderboard illusion," *arXiv preprint*, 2025.

[108] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" *arXiv preprint*, 2025.

[109] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics (TACL)*, 2023.

[110] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[111] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.06963.

[112] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis *et al.*, "Advances and open problems in federated learning," *arXiv preprint arXiv:1912.04977*, 2019, published in Foundations and Trends in Machine Learning, vol. 14, no. 1–2, 2021.

[113] T. Guo, S. Guo, J. Wang, and W. Xu, "PromptFL: Let federated participants cooperatively learn prompts instead of models – federated learning in age of foundation model," *arXiv preprint arXiv:2208.11625*, 2022.

[114] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," *arXiv preprint*, 2025.

[115] R. Tiwari, K. Sareen, L. A. Agrawal, J. E. Gonzalez, M. Zaharia, K. Keutzer, I. S. Dhillon, R. Agarwal, and D. Khatri, "Learning, fast and slow: Towards LLMs that adapt continually," *arXiv preprint*, 2026.

[116] N. Vassilyev, W. Berrios, R. Zhang, B. Han, D. Kiela, and S. Mehri, "Reflective context learning: Studying the optimization primitives of context space," *arXiv preprint*, 2026.

[117] Q. Cai, Y. Zhang, X. Jia, H. Zheng, W. Xue, J. Song, X. Tian, and Y. Guo, "MOSS: Self-evolution through source-level rewriting in autonomous agent systems," *arXiv preprint*, 2026.

[118] L. Li, J. Li, J. Zhao, K. Ye, Y. Li, C.-z. Xu, and X. Gao, "Grow the harness, not the context: From strategy-free scaffolds to reusable specialist agents," *arXiv preprint arXiv:2609.26760*, 2026.

[119] M. M. Tanjim, J. Subramanian, X. Chen, B. Kveton, S. Mukherjee, A. Zhang, S. Kim, S. Sarkhel, and S. Choudhury, "MOCHA: Multi-objective chebyshev annealing for agent skill optimization," *arXiv preprint*, 2026.

[120] Y. Shalev, Z. Ding, and M. Jamnik, "Training language agents to learn from experience," *arXiv preprint*, 2026.

[121] H. Li, R. He, Q. Zhang, C. Ji, Q. Mang, X. Chen, L. A. Agrawal, W.-L. Liao, E. Yang, A. Cheung, J. Zou, K. Olukotun, I. Stoica, and J. E. Gonzalez, "Combee: Scaling prompt learning for self-improving language model agents," *arXiv preprint*, 2026.

[122] P. Hebbar, Y. Manawat, S. Verboomen, A. Ivanova, S. Palanimalai, K. Bhatia, and V. Baskaran, "Sia: Self improving ai with harness & weight updates," *arXiv preprint arXiv:2605.27276*, 2026.

[123] R. Kirk, I. Mediratta, C. Nalmpantis *et al.*, "Understanding the effects of RLHF on LLM generalisation and diversity," in *ICLR 2024*, 2024.

[124] V. Padmakumar and H. He, "Does writing with language models reduce content diversity?" *arXiv preprint*, 2023.

[125] S. Dohare *et al.*, "Loss of plasticity in deep continual learning," *Nature*, vol. 632, no. 8026, pp. 768–774, 2024.

[126] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[127] M. L. Arbuzov, S. Bei, Z. Dong, D. Kalaev, and A. A. Shvets, "Beyond exponential decay: Rethinking error accumulation in large language models," *arXiv preprint*, 2025.

[128] M. L. Arbuzov, L. Mosbacker, S. Bei, Z. Dong, D. Kalaev, and A. Shvets, "The architecture of errors: From universal impossibility to patch-local LLM reliability," *arXiv preprint arXiv:2605.30628*, 2026.

[129] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan *et al.*, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, 2026, arXiv:2507.21046v4, 16 Jan 2026.

[130] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv preprint arXiv:2508.07407*, Aug. 2025.

[131] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, "Self-evolving coding agents," *arXiv preprint arXiv:2608.03392*, Aug. 2026.

[132] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, "Always-on agents: A survey of persistent memory, state, and governance in LLM agents," *arXiv preprint arXiv:2606.30306*, 2026.

[133] Z. Lin, X. Hao, R. Fu, S. Cui, K. Chen, C. Li, Z. Li, and F. Xiong, "A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle," in *Proceedings of EMNLP 2026*, 2026, arXiv:2604.16548v3, 22 Sep 2026.

[134] J. Guo, Z. Hao, C. Wang, C. Fan, T. Luo, H. Li, Y. Gao, H. Mei, J. Peng, R. Xu, M. Dong, H. Wu, M. Zheng, K. Han, S. Wang, C. Xu, and Y. Wang, "From question answering to task completion: A survey on agent system and harness design," *arXiv preprint arXiv:2606.20683*, Jun. 2026.

[135] B. Xia, L. Zhu, Z. Xing, Q. Lu, D. Sejdinovic, and X. Xu, "Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents," *arXiv preprint arXiv:2606.20631*, May 2026.

[136] K. Ding, Y. Zhou, C. Jin, F. Tong, M. Zhou, and D. N. Metaxas, "Agent skill evaluation and evolution: Frameworks and benchmarks," *arXiv preprint arXiv:2606.11435*, Jun. 2026.

[137] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," *arXiv preprint arXiv:2507.13334*, Jul. 2025.

[138] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.03409.

[139] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.08532.

[140] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[141] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[142] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic "differentiation" via text," *arXiv preprint*, 2024.

[143] Q. Zhang, C. Hu, S. Upasani *et al.*, "Agentic context engineering: Evolving contexts for self-improving language models," arXiv preprint arXiv:2510.04618, 2025.

[144] W. Zhou, Y. Ou, S. Ding, L. Li, J. Wu, T. Wang, J. Chen, S. Wang, X. Xu, N. Zhang, H. Chen, and Y. E. Jiang, "Symbolic learning enables self-evolving agents," *arXiv preprint arXiv:2406.18532*, 2024.

[145] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," in *First Conference on Language Modeling (COLM 2024)*, 2024, arXiv:2310.02304.

[146] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint arXiv:2504.15228*, 2025.

[147] Q. Yang, Z. Kou, X. Zhang, D.-D. Wu, H. Gu, J. Guo, Y. Liu, D. Jiang, and Q. Xu, "Federated agent optimization," *arXiv preprint arXiv:2610.01195*, 2026.

[148] H. Robbins and D. Siegmund, "A convergence theorem for non negative almost supermartingales and some applications," in *Optimizing Methods in Statistics*. Academic Press, 1971, pp. 233–257.

MIKHAIL L. ARBUZOV TODO(bio).

LEE MOSBACKER TODO(bio).

SISONG BEI TODO(bio).

ZIWEI DONG TODO(bio).

DMITRI KALAEV TODO(bio).

ALEXEY SHVETS TODO(bio).